Motqin Learning Session Algorithm — Specification v1.0

Status: authoritative. Both the React (web) and Flutter (mobile) implementations must conform to this document. Any behavior change requires a version bump here **and** updated test vectors (§9). If an implementation and this spec disagree, the spec wins; if the spec is ambiguous, fix the spec first, then the code.

---

1. Scope

This spec defines the **in-session queue algorithm**: which card the student sees next during an active study session, and how their answers change the queue. It does **not** cover spaced repetition across sessions, UI/animation, or answer persistence beyond the reporting contract in §8.

The algorithm is a **pure, deterministic function**. Given the same item list, the same configuration, and the same sequence of events, both platforms MUST produce the exact same sequence of cards. There is no randomness anywhere in the core algorithm (§7.4).

---

2. Configuration constants

Constants are delivered to the client (config file or remote config) — never hard-coded in the algorithm body.

Constant	v1.0 value	Meaning
`BATCH_SIZE`	6	Maximum number of items in the active rotation at once
`GRADUATE`	3	Mastery level at which an item retires from the session
`MIN_GAP`	2	Minimum number of *other* cards between two showings of the same item

Constraints: `BATCH_SIZE >= 1`, `GRADUATE >= 1`, `MIN_GAP >= 0`. Note that `MIN_GAP` must be `<= BATCH_SIZE - 1` for the spacing rule to be satisfiable when the rotation is full; the fallback rule (§6 step 4) handles the endgame where it is not.

---

3. Input: the session payload

The client fetches `GET /api/lessons/{id}/session` once, before the session starts. The server returns items **already sorted** by `DisplayOrder` ascending, then `Priority`

ascending (1 = High first), then `QuestionID` ascending.

The client MUST NOT re-sort. The array index in the payload is the item's `queueIndex` and is the final tie-breaker everywhere in this spec. Keeping the sort server-side means there is exactly one implementation of the ordering rule.

Each item carries:

```
SessionItem {
    questionId:   int
    questionType: "MultipleChoiceQuestion" | "FillInTheBlankQuestion"

    // presentation (information) fields
    title:        string | null
    description:  string          // the teaching content shown on the
presentation card
    imageUrl:     string | null
    audioUrl:     string | null

    // exercise fields
    questionText: string
    answerOptions: string | null   // MCQ only, comma-joined
    correctAnswer: string | null   // MCQ only
    correctText:   string | null   // FIB only
    caseSensitive: bool | null     // FIB only
}
```

---

4. State

All session state lives in two plain-data structures. There is one `ItemState` per question in the payload, and one `SessionState` for the whole session. Nothing else may hold algorithm state.

```
ItemState {
    queueIndex:      int          // index in the payload array, fixed
    introduced:      bool         // presentation card has been shown
(init: false)
    retired:         bool         // graduated, out of the session
(init: false)
    mastery:         int          // 0..GRADUATE
(init: 0)
    lastShownAtTurn: int | null   // turn of most recent card of ANY kind
(init: null)
}
```

Field by field:

`queueIndex` — the item's position in the payload array (0-based). Set once at `init()` and never changed. It has two jobs: it encodes the author's intended teaching order (the server sorted by `DisplayOrder` → `Priority` → `QuestionID`, §3), so step 2 of `next()` introduces items in `queueIndex` order; and it is the *final* tie-breaker in step 5, which is what makes selection fully deterministic — two items can never be "equally choosable."

`introduced` — `false` until the item's presentation card is issued for the first time, then `true` forever (a forced re-teach in §6 step 1 shows the presentation card again but does not touch this flag — the item is already introduced). An item can never receive a `TestCard` while `introduced == false`: information always precedes testing.

`retired` — `false` until the item graduates (its mastery reaches `GRADUATE` on a correct answer, §7.2), then `true` forever. A retired item is out of the session: it can never be selected again by any step of `next()`. Retiring an item is what frees a rotation slot and lets step 2 introduce the next unintroduced item.

`mastery` — the item's progress counter, starting at 0. Correct answer: `+1`. Wrong answer: `-1`, floored at 0. It never exceeds `GRADUATE` (the item retires the moment it gets there). Mastery drives two things only: *who is tested next* (step 5 prefers the lowest mastery, so the weakest items get the most repetitions) and *graduation*. It does **not** change the exercise type (§5).

`lastShownAtTurn` — the turn number on which this item's card was most recently issued, of **either** kind: presentation and test cards both count, including forced re-teaches. `null` means "never shown," which only ever holds before the item is introduced. It is written at card-issue time inside `next()` (§6), never inside the answer handler (§7.2). Its two consumers are the spacing rule in step 4 (an item shown at turn `t` is not testable again until at least `MIN_GAP` other cards have appeared) and the "least recently shown" tie-break in step 5, where `null` sorts before every number.

```
SessionState {
    turn:              int                 // init: 0, see below
    items:             ItemState[]         // same order as payload
    forcedPresentation: questionId | null  // init: null, see §6 step 1
    currentCard:       Card                // output of the last next() call
}
```

`turn` — a global counter of cards issued, starting at 0 so that the first card is turn 1. Every `PresentationCard` and every `TestCard` increments it by exactly 1; the terminal `SummaryCard` does **not** (it is a result screen, not a scheduling event). Turns are the session's clock: all spacing math in step 4 is expressed in turns, not wall time.

`items` — the array of `ItemState`, in the same order as the payload, so `items[i].queueIndex == i` always holds. Implementations may keep a `questionId → index` map for convenience, but the array is the state.

`forcedPresentation` — normally `null`. When a student answers wrong, §7.2 sets it to that item's `questionId`; step 1 of the very next `next()` call sees it, issues that item's presentation card again ("you missed it — here is the information again"), and resets the field to `null`. It holds at most one id: it is written on a wrong answer and consumed by the immediately following `next()`, so a second wrong answer cannot occur before it is cleared.

`currentCard` — the card the UI is currently displaying, i.e. the return value of the most recent `next()`. Kept in state for two reasons: event validation (§7 — `CONTINUE` is only legal on a presentation card, `ANSWER` only on a test card) and local pause/resume (§10), which serializes `SessionState` and must restore the exact card the student was looking at.

Shorthand definitions used in the rest of the spec:

- **active item** — `introduced && !retired`: it is in the rotation, being tested toward graduation. At most `BATCH_SIZE` items are ever active at once — step 2 only introduces a new item when the active count is below `BATCH_SIZE`.
- **unintroduced item** — `!introduced`: still waiting in the queue; the student has not seen it in any form.
- (A third implicit category is **retired**; every item is always exactly one of the three, and moves only forward: unintroduced → active → retired.)

---

5. Card types

```
PresentationCard { item }   // shows title, description, image, audio. No
answering.
                            // Advanced by the CONTINUE event.

TestCard { item }           // renders the item's native exercise (MCQ or FIB).
                            // Advanced by the ANSWER event.

SummaryCard { stats }       // terminal. Session is over. See §8.3 for stats.
```

There is no exercise-type escalation: an item is always tested in its **native type** from the payload. Mastery affects only scheduling and graduation.

---

6. The `next()` function

Called to produce each card. Steps are evaluated **strictly in this order**; the first step that produces a card wins.

```
next(state):
    T = state.turn + 1                          // the turn this card will occupy

    // STEP 1 — forced re-teach (highest priority)
    if state.forcedPresentation != null:
        item = the forced item
        state.forcedPresentation = null
        item.lastShownAtTurn = T
        state.turn = T
        return PresentationCard(item)

    // STEP 2 — refill the rotation
    activeCount = count of active items
    if activeCount < BATCH_SIZE and any unintroduced item exists:
        item = unintroduced item with the LOWEST queueIndex
        item.introduced = true
        item.lastShownAtTurn = T
        state.turn = T
        return PresentationCard(item)

    // STEP 3 — session complete
    if activeCount == 0:
        return SummaryCard(stats)               // turn does NOT increment

    // STEP 4 — choose an item to test
    eligible = active items where lastShownAtTurn == null
                              or (T - lastShownAtTurn) > MIN_GAP
    if eligible is empty:
        // endgame fallback, tier 1: everything except the most recently shown
        eligible = active items where lastShownAtTurn != T - 1
    if eligible is empty:
        // tier 2: unavoidable immediate repeat (e.g. one active item left)
        eligible = all active items

    // STEP 5 — deterministic selection
    item = min of eligible by:
        (1) lowest mastery
        (2) then lowest lastShownAtTurn   (null sorts FIRST, i.e. before any
number)
        (3) then lowest queueIndex

    item.lastShownAtTurn = T
    state.turn = T
    return TestCard(item)
```

Notes on intent, for reviewers: step 1 before step 2 means a failed item is re-taught *immediately*, even if a slot is free for a new item — re-teaching beats introducing. The spacing rule in step 4 reads as "at least `MIN_GAP` other cards since this item was last on screen." With `MIN_GAP = 2`, an item shown at turn 5 is eligible again at turn 8 or later.

---

7. Events

Exactly two events advance the session. Both end by calling `next()`.

7.1 `CONTINUE`

Valid only while `currentCard` is a `PresentationCard`. No state change other than advancing. (The student read the information and tapped continue.)

7.2 `ANSWER { correct: bool }`

Valid only while `currentCard` is a `TestCard`, for the item on that card.

```
onAnswer(state, item, correct):
    if correct:
        item.mastery = item.mastery + 1
        if item.mastery >= GRADUATE:
            item.retired = true              // slot freed; refill happens in
next()
    else:
        item.mastery = max(0, item.mastery - 1)
        state.forcedPresentation = item.questionId
    // lastShownAtTurn was already set when the TestCard was issued — do not
touch it here
```

7.3 Answer correctness (client-side check)

The client determines `correct` before raising `ANSWER`:

- **MCQ** — the selected option string equals `correctAnswer` exactly (after §7.5 normalization of both sides).
- **FIB** — the typed text equals `correctText` after §7.5 normalization; if `caseSensitive == false`, compare case-insensitively using simple Unicode case folding. (Arabic has no case; this flag matters for English/Latin content.)

7.4 Determinism rules

The core algorithm contains **no random number generator**. Shuffling MCQ answer options for display is a **UI concern**, done outside the algorithm, and MUST NOT affect state or correctness checking (compare option *values*, not positions). This is what makes the shared test vectors (§9) portable across platforms.

7.5 Text normalization (v1.0)

Applied to both the user's input and the stored answer before comparison: trim leading/trailing whitespace; collapse any internal run of whitespace to a single space. Nothing else in v1.0 — no Arabic diacritic stripping, no alef/hamza normalization. (Flagged as a likely v1.1 addition; when it lands, it lands in this section for both platforms at once.)

---

8. Server reporting contract

Reporting is **fire-and-forget with retry**: a failed report never blocks the next card. Queue failed reports and retry on reconnect, preserving order per question.

8.1 When a `TestCard` is issued

`POST /api/questions/{id}/start` with `{ sessionId, startTime }` (ISO 8601 UTC).

8.2 When `ANSWER` is raised

`POST /api/questions/{id}/end` with `{ sessionId, endTime, userAnswer, isCorrect }`.

Presentation cards are not reported in v1.0.

8.3 Summary stats (client-computed, shown on `SummaryCard`)

Per item: attempts, correct, wrong. Totals: items graduated, total test cards, total presentation cards (including forced re-teaches), accuracy = correct / total test cards, session duration.

---

9. Conformance test vector

Every implementation MUST reproduce this trace exactly. Config for the vector (test-only values, not production): `BATCH_SIZE = 2`, `GRADUATE = 2`, `MIN_GAP = 1`. Payload: three items A, B, C with `queueIndex` 0, 1, 2 (`questionId` 101, 102, 103).

Scripted events, in order, with the expected card **before** each event:

Turn	Expected card	Event	State after event
1	Present(A)	CONTINUE	A introduced, A.last=1
2	Present(B)	CONTINUE	B introduced, B.last=2
3	Test(A)	ANSWER correct	A.mastery=1
4	Test(B)	ANSWER wrong	B.mastery=0, forced=B
5	Present(B)	CONTINUE	forced cleared, B.last=5
6	Test(A)	ANSWER correct	A.mastery=2 → A retired
7	Present(C)	CONTINUE	C introduced, C.last=7
8	Test(B)	ANSWER correct	B.mastery=1
9	Test(C)	ANSWER correct	C.mastery=1
10	Test(B)	ANSWER correct	B.mastery=2 → B retired
11	Test(C)	ANSWER correct	C.mastery=2 → C retired
—	Summary	—	totals: 8 test cards, 7 correct, 1 wrong

Why each choice falls out of §6, spot-checking the non-obvious turns: turn 3 — batch full (A,B active); B was shown at turn 2, so `3-2=1` is not `> MIN_GAP`; only A eligible. Turn 4 — A ineligible (`4-3=1`), B eligible (`4-2=2`). Turn 5 — step 1 fires (forced=B) even though no slot is free. Turn 6 — B ineligible (`6-5=1`); A eligible and tested, graduates. Turn 7 — step 2 fires: active count 1 < 2 and C is unintroduced. Turn 11 — C is the only active item; `11-9=2 > 1` so it is eligible normally, no fallback needed.

A second micro-check for the fallback tiers: if the session ever reaches a single active item that was shown on the immediately previous turn, tier 1 excludes it, comes up empty, and tier 2 shows it again back-to-back. This is the only situation where an immediate repeat is permitted.

---

10. Edge cases (defined behavior)

An **empty payload** (lesson with zero questions) initializes directly to `SummaryCard` with zero stats — though the server should have returned 404/empty earlier. `BATCH_SIZE` **larger than the item count** is fine: step 2 simply introduces everything, then the rotation runs below capacity. **A student who keeps failing** never graduates the item and the session has no upper bound on length; the UI must always offer an explicit "end session" action, which jumps straight to the summary with current stats. **Pause/resume on the same device** is

done by serializing `SessionState` locally (it is plain data); cross-device resume is out of scope for v1.0 by design — a session restarts from the beginning on another device.

---

11. Implementation requirements (both platforms)

Implement the algorithm as a pure module — `init(items, config) → state`, `next(state) → (state, card)`, `apply(state, event) → state` — with **zero imports** from UI, networking, or platform APIs. Networking (§8) subscribes to the module's outputs; it is not called from inside it. Port the conformance vector in §9 as an automated test in each codebase; CI must run it. The two source files (TS and Dart) should each carry a header comment: `// Conforms to Session Algorithm Spec v1.0 — do not change behavior without bumping the spec.`